SWGL -- Making a weapon from a 3D model

2026-10-11

What a weapon is made of

A SWGL weapon is a `.wpn` file plus three models and some assets, all in one `Z_*.pk3`. No code change is needed: the game reads every `ext_data/*.wpn` at startup and builds the weapon, its pickup item and its ammo from it. The reference example is `C:\Program Files (x86)\Steam\steamapps\common\Jedi Academy - SWGL_DEV\GameData\SWGL\Z_Halo_plasma_pistol.pk3` (class `weapon_plasma_pistol`).

> **A `baseweapon` is mandatory.** Every weapon must declare `baseweapon <existing hardcoded class>` in its `.wpn`. The engine inherits every field you leave out from it, and also uses it for hardcoded behaviour (charging glow, throwables, NPC AI) and to index per-weapon tables (hit effects, firing logic). A weapon without a `baseweapon` has no base class: missing fields are 0 or empty, and some code reads tables out of bounds. Always start from the stock weapon that plays closest to yours.

File (example)	Role	Required
`ext_data/weapon_plasma_pistol.wpn`	Weapon definition: model, icon, attacks, effects, sounds. Inherits everything else from `baseweapon`.	Yes
`models/weapons2/plasma_pistol/plasma_pistol.md3`	First-person (view) model: the gun mesh and the first-person arm, plus `tag_flash` (muzzle).	Yes
`models/weapons2/plasma_pistol/plasma_pistol_hand.md3`	Tags only, 15 frames: `tag_weapon` places the view model in front of the camera.	No -- falls back to `briar_pistol_hand.md3`
`models/weapons2/plasma_pistol/plasma_pistol_w.glm`	Third-person / pickup model (Ghoul2), with `*flash` and `*weapon` tags.	Yes
`models/weapons2/plasma_pistol/plasma_pistol.tga` (or `.jpg` / `.png`)	Texture, named like the shader path on the surfaces.	Yes
`gfx/hud/w_icon_plasma_pistol.tga` + `_na.tga`	Weapon icon, and the "no ammo" variant (loaded automatically).	Yes
`effects/plasma_pistol/*.efx`	Muzzle flash and projectile effects.	No -- inherited

File (example)	Role	Required
`sound/weapons/plasma_pistol/*`	Fire / charge sounds (the fire sound is usually played from the muzzle `.efx`).	No -- inherited
`shaders/plasma_pistol.shader`	Shaders for the effect sprites.	Only for custom effects
`strings/English/weapon_plasma_pistol.str`	Display name (`NAME`) and datapad text (`DESC`).	Recommended

How the models attach in game:

```
First person                                Third person
camera                                      character
  +-- _hand.md3                               +-- *weapon bolt (hand)
        +-- tag_weapon                              +-- plasma_pistol_w.glm
              +-- plasma_pistol.md3  (built)              +-- *flash (fires)
                    +-- tag_flash (fires)
```

In first person the camera carries the hand, the hand's `tag_weapon` carries your view model, and its `tag_flash` fires the shots. In third person the character's `*weapon` bolt carries the `_w.glm`, and its `*flash` tag fires. The two models you build are the view model and the `_w.glm`.

All file names derive from the `weaponmodel` path: strip `.md3` and add `_hand.md3`, `_w.glm`, `_barrel.md3`. The engine code that does this is `CG_RegisterWeapon` and `CG_GetWorldModelName` in `cg_weapons.cpp`.

1. Prepare the model in Blender

Work in game units: 1 Blender unit = 1 game unit (about 1 inch; a player is about 64 units tall). The example pistol is about 19 units long. Quake axes match Blender: +X is forward (the barrel points along +X), +Z is up.

Tools you need:

- **Blender Jedi Academy Plugin Suite** (v2.1.0, Blender 4.1+): GLM import/export for the world model.
- An **MD3 importer/exporter** for the view model and the `_hand.md3`. The JA suite no longer has one; its manual points to SomaZ's Blender_BSP_Importer, which reads and writes MD3 with tags as empties.

Prepare the mesh:

1. Import the source model, apply all transforms (Ctrl+A > All Transforms), and scale it to game units.
2. Put the grip at the origin and point the barrel along +X. The origin is where the hand holds the gun.
3. Triangulate, and keep each mesh under 1000 vertices (engine limit per surface). Split larger guns into several objects.
4. Give each mesh one material. For MD3 and GLM the material / shader name is the texture path without extension, e.g. `models/weapons2/plasma_pistol/plasma_pistol`. The engine then finds `plasma_pistol.tga`, `.jpg` or `.png`, or a shader of that name.
5. Bake the texture to one image if the source has several (fewer surfaces = fewer draw calls).

For the view model (`.md3`):

- Add an empty named `tag_flash` at the muzzle tip, its X axis along the barrel. The game spawns the muzzle effect and the projectile from it.

- Keep an empty `tag_weapon` at the origin with no rotation (the example has one; it is the attach point).
- Add the first-person arm as a second mesh. The Halo example reuses the arm of the stock blaster (`hand_arm` surface, shader `models/weapons2/blaster_r/forearm.jpg`). Import a stock view model (e.g. `models/weapons2/blaster_r/blaster.md3`) and copy its arm, then pose it to hold your grip.

For the world model (`_w.glm`):

- Same mesh, no arm. Import a stock `_w.glm` (e.g. `models/weapons2/blaster_pistol/blaster_pistol_w.glm`) as a reference and match its grip position and orientation -- the GLM is bolted to the character's `*weapon` hand bolt.
- Hierarchy required by the exporter: `scene_root` (empty) > `model_root_0` (empty) > one root mesh > other meshes and tags.
- Use the "Ghoul 2 Properties" panel (Object tab) on every object: set its Name, tick Tag for tags.
- Tags are one-triangle meshes named `*flash` (muzzle, used for the 3rd-person muzzle point) and `*weapon` (used by some NPC AI). Vertex 2 is the origin, vertex 0 gives +X, vertex 1 gives +Y.
- No armature is needed: leave the skeleton out and the exporter uses `*default`. Unweighted vertices go to the root bone.

2. Preview the first-person view in Blender

The game draws the view model like this: the camera is at the origin looking along +X; the `_hand.md3` sits at the camera; the view model is placed on the hand's `tag_weapon`. You can rebuild that in Blender and see the in-game framing before you export.

Pick a `_hand.md3` first. Copy the one of a similar stock weapon and rename it after your model (`plasma_pistol_hand.md3`). For a pistol use `models/weapons2/blaster_pistol/blaster_pistol_hand.md3` or `briar_pistol/briar_pistol_hand.md3`; for a rifle `blaster_r/blaster_hand.md3`. It holds only tags (`tag_weapon`, `tag_flash`), animated over 15 frames.

1. Import your view model (MD3 importer, or keep your working scene).
2. Import the `_hand.md3`. Its tags come in as empties; Blender renames the duplicate to `tag_weapon.001`.
3. Parent your `tag_flash` to the gun mesh, and the gun mesh to your `tag_weapon` (the one at the model origin).
4. On your `tag_weapon`, add a Copy Transforms constraint with the hand's `tag_weapon` as target. The gun now sits where the game draws it.
5. Add a camera at location 0, 0, 0, rotation mode XYZ Euler, rotation 90, 0, 270. It now looks along +X with +Z up, like the game.
6. Camera > Lens: Field of View type, Sensor Fit: Horizontal, FOV 80° (the default `cg_fov`). Use your own `cg_fov` if you changed it.
7. Output resolution 4:3 (e.g. 1024 x 768). With `cg_fovAspectAdjust 1`, 80° is defined for 4:3; with the default `cg_fovAspectAdjust 0` the horizontal FOV stays 80° at any aspect, so you can also use your screen aspect.
8. Numpad 0 for the camera view. Scrub frames 0-14 to check the poses.

What the hand frames mean (from `CG_MapTorsoToWeaponFrame`):

Frames	Pose
0	Idle / ready
1-6	Firing
6-10	Lowering (weapon switch out)

Frames	Pose
11-14	Raising (weapon switch in)

In game the gun moves further only from `cg_gunX/Y/Z` (default 0), lean, and a drop when FOV is above 90 with `cg_fovViewmodelAdjust 1`. To tune the placement, move the gun mesh relative to your `tag_weapon`, not the tag itself, and re-export.

3. Export the models

Work inside a folder that mirrors the pk3, e.g. `MyWeapon/models/weapons2/plasma_pistol/`. Every path in the files is relative to the pk3 root.

View model -- `plasma_pistol.md3`

1. Remove the hand empties and the camera from the selection (or delete them after the preview).
2. Select the gun mesh, the arm mesh, `tag_flash` and `tag_weapon`.
3. Export with the MD3 exporter, one frame. Surface shader = the material name (`models/weapons2/plasma_pistol/plasma_pistol`).

Hand -- `plasma_pistol_hand.md3`

The copied stock file is enough. Edit and re-export it only to change the hold pose or the swing (keep 15 frames and the tag names).

World model -- `plasma_pistol_w.glm`

File > Export > JA Ghoul 2 model (.glm), with:

Setting	Value
Base Path	Your mirror folder (e.g. `C:/MyWeapon`), or empty if the path contains `GameData`
.gla name	Empty (weapons have no skeleton; the file gets `*default`)

The exported file name must be the view model name with `_w.glm`: the engine strips `.md3` and adds `_w.glm` (unless the name already contains `_w`).

Optional -- barrels

For parts that spin or move on their own (e.g. a rotary cannon), set `barrelcount N` in the `.wpn` and provide `plasma_pistol_barrel.md3`, `_barrel2.md3`, ... (max 4). They attach to `tag_barrel`, `tag_barrel2`, ... in the `_hand.md3`.

Check before you go on

- Open the GLM in ModView or re-import it: one surface per mesh, `*flash` and `*weapon` present.
- Re-import the MD3: `tag_flash` X axis points out of the barrel.

4. Declare the weapon (.wpn)

Create `ext_data/<weaponclass>.wpn`. The game loads `ext_data/weapons.dat` first, then every `.wpn` (code: `WP_LoadWeaponParms` in `g_weaponLoad.cpp`). A new `weaponclass` gets a new weapon slot; an existing one

overrides that weapon.

The example, complete:

```
{
    weaponclass         weapon_plasma_pistol
    baseweapon          weapon_blaster_pistol
    weaponmodel         models/weapons2/plasma_pistol/plasma_pistol.md3
    weaponIcon          gfx/hud/w_icon_plasma_pistol
    attackDefinition main
    {
        muzzleEffect        plasma_pistol/muzzle_flash
        projectileEffect    plasma_pistol/shot
    }
    attackDefinition alt
    {
        chargesound         sound/weapons/plasma_pistol/charge.mp3
        chargeMuzzleShader  gfx/effects/greenFrontFlash
        muzzleEffect        plasma_pistol/muzzle_flash
        projectileEffect    plasma_pistol/shot
    }
}
```

`baseweapon` **is mandatory. Never leave it out.** It copies every field you do not set from that weapon: damage, fire rate, ammo, firing logic, sounds, effects, animations. It must be one of the hardcoded classes, because the game also uses it for hardcoded behaviour (charging glow, throwables, NPC AI) and for per-weapon lookup tables. Without it, the weapon starts empty (all 0) and some tables are read out of bounds. Pick the stock weapon that plays closest to yours, then override only what differs. Base classes include `weapon_blaster_pistol`, `weapon_bryar_pistol`, `weapon_blaster`, `weapon_disruptor`, `weapon_bowcaster`, `weapon_repeater`, `weapon_demp2`, `weapon_flechette`, `weapon_rocket_launcher`, `weapon_concussion_rifle`, `weapon_thermal`, `weapon_clonerifle`, `weapon_clonepistol`, `weapon_rebelrifle` (full list: `_weaponIndexes` in `g_weaponLoad.h`). Throwables based on `weapon_thermal`, `weapon_det_pack` or `weapon_trip_mine` get their own ammo item automatically.

Weapon fields (most used):

Field	Value
`weaponclass`	Unique id, should start with `weapon_` (max 31 chars). Also the string key prefix and the item name.
`baseweapon`	**Required.** Class to inherit from.
`weaponmodel`	View model `.md3` path (max 63 chars). Derives `_hand.md3`, `_w.glm`, `_barrel.md3`.
`weaponIcon`	HUD icon path without extension; `<icon>_na` is the no-ammo variant.
`weaponCategory`	Hold animation set: `WC_PISTOL`, `WC_LIGHT`, `WC_HEAVY`, `WC_SNIPER`, `WC_MINIGUN`, `WC_SHOULDER`, `WC_GRENADE`, `WC_EXPLOSIVE`, `WC_MELEE`, `WC_MELEE_1H`, `WC_STUN_BATON`.
`weaponBucket`	Selection group: `WB_PISTOLS`, `WB_BLASTERS`, `WB_SPECIALISTS`, `WB_HEAVY_WEAPONS`, `WB_MELEE`, `WB_THROWABLES`, `WB_OTHERS`.
`ammotype`, `ammolowcount`, `ammomax`	Ammo pool, low-ammo warning, max carried (up to 1000).
`selectsound`, `readySound`, `stopsound`	Sounds on switch in / ready / stop.
`barrelcount`	0-4 extra barrel models.
`playerUsable`	0 to make it NPC only.

Field	Value
`scopeType`, `scopeFov`, `scopeMask`, `scopeInsert`, `scopeFullMask`	Scope: `ST_DISRUPTOR`, `ST_PROGRESSIVE`, `ST_A280`, `ST_DC17M`, `ST_E5`, `ST_EE3`, `ST_F11D`, `ST_CUSTOM`.

Full field list in the Weapon Fields appendix.

Attacks. Up to four `attackDefinition` blocks: `main`, `alt`, `scoped_main`, `scoped_alt`. A scoped block with no `attackType` takes the type of its main/alt attack. Most used fields:

Field	Value
`attackType`	Firing logic, e.g. `FL_BLASTER`, `FL_BLASTER_CHARGED`, `FL_BOWCASTER`, `FL_BEAM`, `FL_FULL_BEAM`, `FL_FLECHETTE`, `FL_MISSILE`, `FL_GRENADE`, `FL_FLAMETHROWER`, `FL_MELEE`.
`damage`, `npcDamage`, `splashDamage`, `splashRadius`	Damage values (`npcDamage` takes 3 values, one per difficulty).
`fireTime`, `energypershot`, `velocity`, `spread`, `range`	Rate (ms), ammo per shot, projectile speed, spread, range.
`chargeUnitTime`, `maxChargeUnits`	Charge-up attacks.
`muzzleEffect`, `projectileEffect`, `explosionEffect`, `hitWallEffect`, `hitFleshEffect`, `hitDroidEffect`	Effect paths under `effects/`, without the `.efx`.
`firingsound`, `startsound`, `chargesound`	Sound paths.
`chargeMuzzleShader`, `chargeMuzzleScale`	Glow at the muzzle while charging.
`beamShader`, `beamColor`, `missileModel`, `missileLight`, `missileLightColor`	Beam / projectile look.
`blockability`	3 values: `B_BLOCKABLE`, `B_DEFLECTABLE` or `B_PASSTHROUGH`.

Cheat command `weaponStat weapon_plasma_pistol` prints the final values after inheritance; use it to check what you actually got.

Effects, icons and text

- **Effects:** `effects/plasma_pistol/muzzle_flash.efx` is referenced as `plasma_pistol/muzzle_flash`. Edit them with EffectsEd, or copy a stock `.efx` and change the shaders. The example plays its fire sound from the muzzle `.efx` (Sound primitive).
- Effect sprites need a shader (`shaders/plasma_pistol.shader`), usually `blendFunc GL_ONE GL_ONE` (additive) and `cull twosided`.
- **Icons:** `gfx/hud/w_icon_plasma_pistol.tga` and `w_icon_plasma_pistol_na.tga`. Match the size of the stock icons.
- **Text:** `strings/English/weapon_plasma_pistol.str`. The file name must be the `weaponclass`; the game looks up `<WEAPONCLASS>_NAME` (pickup / selection name) and `<WEAPONCLASS>_DESC` (datapad and loadout text). Copy the example file and change the two `LANG_ENGLISH` lines.

5. Package and test

1. Zip the mirror folder's contents (so `ext_data/`, `models/`, ... are at the zip root) and rename it `Z_<something>.pk3`.

2. Put it in `GameData/SWGL/`. The `Z_` prefix makes it load after `SWGL_Weapons.pk3` and `Z_SWGL_Weapons.pk3`, so your files win on a name clash.
3. Start SWGL single player and load a map with cheats: `devmap <map>`.
4. Check the console at startup: `Found N External files` then `Parsing weapon_plasma_pistol.wpn`. Any `WARNING: weapon 'weapon_plasma_pistol': ...` line names a bad field.
5. Give the weapon: `give weapon_plasma_pistol`. If the console says `unknown item`, use `give weapons` (all player-usable weapons, dynamic ones included).
6. Check in first person: grip in the hand, arm in view, muzzle flash at the barrel tip, projectile leaving the barrel.
7. Check in third person (`cg_thirdPerson 1`): gun in the hand, muzzle flash at the tip (`*flash`).
8. Drop it or look at a pickup to check the world model on the ground.
9. `weaponStat weapon_plasma_pistol` to read the final damage, fire time and effects.

To iterate quickly, keep the files loose in a test folder and pack only at the end, or restart the game after replacing the pk3 -- the `.wpn` files are read only at startup.

Common problems

Symptom	Cause	Fix
Error `Couldn't find weapon model '...'`	`weaponmodel` path wrong or file missing.	Check the path and case; it is relative to the pk3 root.
Weapon has no damage / wrong behaviour, or crashes	No `baseweapon` in the `.wpn`.	Add `baseweapon` with a hardcoded class. It is mandatory.
Gun floats, sits too low, or points sideways in first person	Grip not at the origin, barrel not along +X, or the wrong `_hand.md3`.	Redo the preview (section 2); move the mesh, not `tag_weapon`.
Shots and flash start from the gun's origin	No `tag_flash` in the view `.md3`.	Add the empty at the muzzle, X along the barrel.
Gun invisible in third person / pickup	`_w.glm` missing or misnamed.	Name it `<weaponmodel without .md3>_w.glm`.
Muzzle flash at the hand in third person	No `*flash` tag in the `_w.glm`.	Add a one-triangle mesh, Ghoul 2 Tag ticked, name `*flash`.
Grey / white-black texture	Surface shader name does not match a texture or shader.	Material name = texture path without extension.
Export error or broken mesh	Surface over 1000 vertices.	Split the mesh.
Weapon name shows as `WEAPON_PLASMA_PISTOL_NAME`	`.str` file name or `REFERENCE` wrong.	File = `strings/English/<weaponclass>.str`, references `NAME` and `DESC`.
Weapon does not exist at all	`.wpn` not read, or a parse error.	Read the startup console; check braces. The combined name list of all `.wpn` files has a 5096-character buffer, so very many add-on files can push yours out.

Symptom	Cause	Fix
Fatal `Error while parsing weapons.dat ... attack definition is badly formated`	Attack name not `main`, `alt`, `scoped_main` or `scoped_alt`, or `{` missing.	Fix the `attackDefinition` line.
MD3 export fails in the JA suite	Its MD3 exporter was removed (the old code needs Blender 2.7).	Use SomaZ's Blender_BSP_Importer for MD3.

Appendix -- Weapon Fields

This is `weaponDefinition.txt`, checked field by field against `g_weaponLoad.cpp` and the code that uses each value. Every key the parser reads is listed, value ranges come from the parser.

Parser rules that apply to every field:

- The key and all its values are on one line. A value on the next line is not read.
- Keys are not case-sensitive. An unknown key prints `WARNING: weapon '<class>': bad parameter ...` and is skipped.
- Paths have no extension for shaders, icons and effects; models and sounds keep theirs. Strings are cut at 63 characters (31 for `weaponClass` and `baseWeapon`).
- A field you leave out is inherited from `baseWeapon`; it is 0 or empty if there is no base weapon -- which is why `baseWeapon` is mandatory.
- `{` and `}` can be anywhere; they do not need their own line.

```
//WP_XXXXX => Comment to find the weapon when a file holds several
{

  weaponClass weapon_xxxxx
         => MUST be the first key: the next keys write into this weapon.
            Start it with "weapon_" (the ammo item is named "ammo_<rest>").
            Max 31 characters. Also the string key: strings/<lang>/<weaponClass>.str, references NAME and DESC.
            An existing class (e.g. weapon_blaster) overrides that weapon; a new one adds a weapon.

  baseWeapon weapon_xxxxx
         => MANDATORY. Every value you leave out is copied from this weapon.
            Use a hardcoded class: the game uses it for hardcoded behaviour (charging glow, throwables, NPC AI).
            With weapon_thermal, weapon_det_pack or weapon_trip_mine an ammo item is generated for your weapon.
            Hardcoded classes: weapon_saber, weapon_blaster_pistol, weapon_blaster, weapon_disruptor,
            weapon_bowcaster, weapon_repeater, weapon_demp2, weapon_flechette, weapon_rocket_launcher,
            weapon_thermal, weapon_trip_mine, weapon_det_pack, weapon_concussion_rifle, weapon_melee,
            weapon_stun_baton, weapon_bryar_pistol, weapon_tusken_rifle, weapon_tusken_staff,
            weapon_noghri_stick, weapon_battledroid, weapon_thefirstorder, weapon_clonecarbine,
            weapon_rebelblaster, weapon_clonerifle, weapon_clonecommando, weapon_rebelrifle, weapon_rey,
            weapon_jango, weapon_boba, weapon_clonepistol, weapon_cis_sniper, weapon_sbd, weapon_droideka
            (NPC / vehicle only: weapon_atst_main, weapon_atst_side, weapon_emplaced_gun, weapon_bot_laser,
            weapon_turret, weapon_tie_fighter, weapon_rapid_concussion, weapon_jawa, weapon_scepter)

  // Models and HUD
  weaponModel models/weapons2/xxx/xxx.md3
         => First-person model (gun + arm, tag_flash). The game derives from it:
              xxx_hand.md3 (tags, falls back to briar_pistol_hand.md3), xxx_w.glm (third person and pickup),
              xxx_barrel.md3, xxx_barrel2.md3 ... (only with barrelCount).

  weaponModel2 models/weapons2/xxx/xxx2.md3
         => Alternative first-person model. It is precached at map load, but no code switches to it now.
            Not needed.

  weaponIcon gfx/hud/w_icon_xxx
```

```
       => HUD and selection icon, no extension. gfx/hud/w_icon_xxx_na is loaded as the "no ammo" icon.

barrelCount 0-4
       => Number of extra barrel models, attached to tag_barrel, tag_barrel2 ... of the _hand.md3.

playerUsable 0/1
       - 0 : NPC only (not given by "give weapons", not in the loadout)
       - 1 : Player can use

weaponCategory
       => The animations the weapon uses (inherited from baseWeapon if left out, else WC_NONE):
       - WC_NONE       : Not really a weapon, blank animations
       - WC_MELEE      : Punching
       - WC_MELEE_1H   : One-handed melee weapon (only tusken_staff uses it)
       - WC_PISTOL     : Pistol animations, can be dual wielded (same as blaster pistol)
       - WC_LIGHT      : When running, held near the leg with one hand (same as blaster)
       - WC_HEAVY      : When running, held at the hip with two hands (same as concussion rifle)
       - WC_GRENADE    : Thermal detonator animations
       - WC_EXPLOSIVE  : Same as det pack and trip mine
       - WC_MINIGUN    : Like a minigun, mainly for the Z6
       - WC_SHOULDER   : Shoulder-carried weapon (rocket launcher style)
       - WC_SNIPER     : When charged, held at the eye
       - WC_STUN_BATON : Held like a light weapon, mainly for the stun baton

weaponBucket
       => Selection group ("bind <key> weapon -X") and order in the weapon list:
       - WB_MELEE         : weapon -1 (saber, melee, stun baton)
       - WB_PISTOLS       : weapon -2 (one-handed weapons)
       - WB_BLASTERS      : weapon -3 (light blasters)
       - WB_SPECIALISTS   : weapon -4 (disruptor, bowcaster, flechette)
       - WB_HEAVY_WEAPONS : weapon -5 (rocket launcher, concussion rifle)
       - WB_THROWABLES    : weapon -6 (grenades and explosives)
       - WB_OTHERS        : weapon -7 (NPC and vehicle weapons)

// Scope
// A scope uses "bind <key> +zoom". scoped_main and scoped_alt are optional: a scoped attack with no
// attackType uses the attackType of main / alt. Define them only to change the attack when zoomed.
// Custom mask and insert: 32-bit TGA (uncompressed recommended), 512x512 or 1024x1024.
// The image is stretched to 4:3: draw it in 4:3, then resize to 1:1. A shader of the same name is optional.
// The insert is drawn behind the mask by default.

scopeType
       - ST_DISRUPTOR   : Progressive zoom, rotating insert. Mask and insert can be overridden. Always full mask.
       - ST_PROGRESSIVE : Progressive zoom, non-rotating insert, custom scopeMask and scopeInsert,
                          configurable full mask.
       - ST_A280        : Static zoom, non-rotating insert, hardcoded mask and insert, never full mask.
       - ST_DC17M       : Static zoom, non-rotating insert, hardcoded mask and insert, never full mask.
       - ST_E5          : Static zoom, non-rotating insert, hardcoded mask and insert, always full mask.
       - ST_EE3         : Static zoom, non-rotating insert, hardcoded mask and insert, never full mask.
       - ST_F11D        : Static zoom, non-rotating insert, hardcoded mask and insert, never full mask.
       - ST_CUSTOM      : Static zoom, non-rotating insert, custom scopeMask and scopeInsert,
                          configurable full mask.

scopeFov <float>
       => Zoom FOV in degrees for static zooms (0 = keep the default). Not used by ST_DISRUPTOR
          and ST_PROGRESSIVE.

scopeMask <path> => Overlay drawn on top of the insert. Stock values:
       - gfx/2d/base_mask   : Non full scope
       - gfx/2d/full_mask   : Full scope (black outside)
       - gfx/2d/cis_mask    : Full scope with red border
       - gfx/2d/cropcircle2 : Full scope used by the disruptor
       - gfx/2d/empty       : Fully transparent

scopeInsert <path> => Reticle drawn inside the scope. Stock values:
       - gfx/2d/a280_insert   : Blue reticle with a red dot
       - gfx/2d/cis_insert    : Small red reticle, inverted "T" and center dot
       - gfx/2d/cropcircle    : Disruptor zoom indicator
       - gfx/2d/dc-17m_insert : Half-transparent light blue/grey reticle with white cross
       - gfx/2d/ee3_insert    : Green reticle (cross + dot) with text around it
       - gfx/2d/f11d_insert   : Three red circles with cross and dot in the center
       - gfx/2d/empty         : Fully transparent

scopeFullMask 0/1
       - 0 : Non full scope
       - 1 : Full scope (HUD hidden, sides black in widescreen)
```

```
// Ammo
ammoType <index> => Ammo pool the weapon uses (number, not a name):
       - 0 : None (infinite)   - 1 : Force        - 2 : Blaster   - 3 : Power cell (repeater)
       - 4 : Metal bolts (bowcaster)              - 5 : Rockets   - 6 : Emplaced
       - 7 : Thermal           - 8 : Trip mine    - 9 : Det pack  - 10 : Flamer
       With baseWeapon weapon_thermal / weapon_det_pack / weapon_trip_mine a new pool is generated:
       leave it out.

ammoLowCount 0-200
       => Ammo count under which the HUD shows the low-ammo warning.

// The next three keys edit an AMMO POOL, shared by every weapon that uses it. weapons.dat uses them in its
// ammo blocks ({ ammo AMMO_BLASTER ammoIcon ... ammoMax ... }). Do not put them in a weapon block.
ammo AMMO_XXX => Select the pool: AMMO_NONE, AMMO_FORCE, AMMO_BLASTER, AMMO_POWERCELL, AMMO_METAL_BOLTS,
                 AMMO_ROCKETS, AMMO_EMPLACED, AMMO_THERMAL, AMMO_TRIPMINE, AMMO_DETPACK, AMMO_FLAMER
ammoIcon <path> => Icon of the selected pool
ammoMax 0-1000  => Maximum ammo carried for the selected pool

// Weapon sounds
selectSound <path> => Played once when you switch to the weapon (default sound/weapons/change.wav).
readySound <path>  => Looped while the weapon is out and not firing (e.g. stun baton hum). Needs a muzzleEffect.
stopSound <path>   => Played once when firing stops. Only works together with readySound.

// Legacy keys, read and ignored: weaponType, firingForce, chargeForce, altChargeForce, selectForce

// Attacks
// attackDefinition main | alt | scoped_main | scoped_alt, then { ... }. Any other name stops the game with
// "an attack definition is badly formated". An attack you leave out is inherited from baseWeapon.
attackDefinition main
{
  attackType => How the attack fires. Some are untested (melee):
  - FL_MELEE            : Bash attack
  - FL_BLASTER          : Simple projectile
  - FL_BLASTER_CHARGED  : Charged shot, like the pistol alt fire
  - FL_BOWCASTER        : Charged spread shot, like the bowcaster main fire
  - FL_BEAM             : Instant beam, like the disruptor main fire
  - FL_FULL_BEAM        : "Intense" beam, adds fullBeamShader on top of beamShader (concussion rifle alt)
  - FL_BEAM_CHARGED     : Charged beam, like the vanilla disruptor scoped fire
  - FL_FLAMETHROWER     : Cone of fire in front of you (uses range, damage, effectDuration)
  - FL_GRENADE_LAUNCHER : Like FL_BLASTER, but the projectile falls
  - FL_DEMP2            : DEMP2 main fire
  - FL_DEMP2_ALT        : DEMP2 alt fire (EMP sphere)
  - FL_FLECHETTE        : Shotgun fire
  - FL_FLECHETTE_ALT    : Flechette alt fire
  - FL_NOGHRI           : Poison attack
  - FL_MISSILE          : Missile
  - FL_MISSILE_AIMED    : Missile with lock-on
  - FL_LASER_TRAP       : Laser trip mine
  - FL_PROXIMITY_TRAP   : Mine that explodes when an enemy is near
  - FL_GRENADE          : Timed grenade
  - FL_IMPACT_GRENADE   : Grenade that explodes on impact
  - FL_STUNBATON        : Republic taser
  - FL_SBD              : Super battle droid wrist blasters (NPC)
  - FL_EXPLOSIVES       : Do not use. For a det pack, use "baseWeapon weapon_det_pack" (still hardcoded).
  - FL_OTHER            : Do not use. NPC hardcoded weapons.
  - FL_NONE             : Empty attack, does nothing.

  // Damage and rate
  damage <int>          => Damage of the player's shot. Charged attacks scale it.
  npcDamage <e> <m> <h> => NPC damage on easy, medium, hard (g_spskill 0/1/2). Boba, Jango and
                           Mandalorians use damage, like the player.
  splashDamage <int>    => Damage to everything inside splashRadius
  splashRadius <float>  => Radius of the splash damage, in units
  energyPerShot 0-1000  => Ammo used per shot
  fireTime 0-10000      => Delay between shots, in ms
  fireOptions FT_XXX <Y> <Z> => Trigger mode:
  - FT_AUTOMATIC    : Hold to fire
  - FT_SEMI         : One click per shot
  - FT_BURST        : Y = shots per burst, Z = delay between the shots of a burst (ms);
                      fireTime is the delay between bursts
  - FT_HIGH_POWERED : One click per shot, damage forced to 200 (HIGH_POWERED_DAMAGE). Use with care.
  methodOfDeath MOD_XXX       => Means of death of a direct hit (kill messages, death reactions, dismemberment)
  splashMethodOfDeath MOD_XXX => Means of death of the splash damage
      e.g. MOD_BRYAR, MOD_BLASTER, MOD_DISRUPTOR, MOD_SNIPER, MOD_BOWCASTER, MOD_REPEATER, MOD_DEMP2,
      MOD_FLECHETTE, MOD_ROCKET, MOD_CONC, MOD_THERMAL, MOD_DETPACK, MOD_LASERTRIP, MOD_MELEE,
```

```
    MOD_CLONERIFLE, MOD_REBELBLASTER, MOD_CLONEPISTOL, MOD_CIS_SNIPER, MOD_ELECTROCUTE, MOD_EXPLOSIVE,
    MOD_ENERGY, MOD_HIGH_POWERED_SHOT ... (most also have an _ALT variant; full list:
    ATK_MethodOfDeathInternal)
blockability <e> <m> <h> => How a saber reacts to the shot on easy, medium, hard:
- B_DEFLECTABLE : The saber sends it back
- B_BLOCKABLE   : The saber stops it
- B_PASSTHROUGH : The saber cannot stop it
    (Below easy, every shot is deflectable; above hard, every shot passes through.)

// Aim
spread <float>        => Player spread. < 1.0 is precise, > 1.5 is not precise, can be 0.
npcSpread <e> <m> <h> => NPC spread on easy, medium, hard. A value left out uses spread.
range 0-...           => Reach of FL_FLAMETHROWER, in units

// Sounds
startSound <path>  => Played once when firing starts
firingSound <path> => Looped while firing (needs a muzzleEffect)
chargeSound <path> => Played while charging
// Most weapons play their shot sound from the muzzleEffect .efx (Sound primitive), not from these keys.

// Effects (path under effects/, no .efx)
muzzleEffect <path>     => Played at tag_flash / *flash on each shot (can contain the shot sound)
projectileEffect <path> => Drawn on the projectile in flight
hitWallEffect <path>    => Impact on world geometry
hitWallEffect2 <path>   => Impact of a mid-power charged shot (FL_BLASTER_CHARGED, FL_BOWCASTER)
hitWallEffect3 <path>   => Impact of a full-power charged shot (FL_BLASTER_CHARGED, FL_BOWCASTER)
hitFleshEffect <path>   => Impact on a creature
hitDroidEffect <path>   => Impact on a droid (optional, uses hitFleshEffect if left out)

// Projectile (FL_BLASTER, FL_GRENADE_LAUNCHER, FL_MISSILE ...)
velocity <float>        => Speed of the player's projectile, in units/s
npcVelocity <e> <m> <h> => NPC projectile speed on easy, medium, hard. 0 or left out uses velocity.
missileModel <path>     => Model drawn for the projectile (optional)
missileSize <int>       => Half size of the projectile hitbox, in units (0 = a point)
missileMass <int>       => Mass of the projectile (optional)
missileSound <path>     => Looped on the projectile in flight (optional)
missileHitSound <path>  => Precached, but no code plays it now
missileLight 0-500      => Radius of the light the projectile emits (0 = none)
missileLightColor <r> <g> <b> => Light colour, each 0-1 (RGB / 255): 1.0 0.0 0.0 is red
missileFuncName <name>  => Client-side projectile drawing. Planned for removal. Values:
    bryar_func, bryar_alt_func, blaster_func, blaster_alt_func, bowcaster_func, repeater_func,
    repeater_alt_func, demp2_func, demp2_alt_func, flechette_func, flechette_alt_func, rocket_func,
    rocket_alt_func, conc_func, emplaced_func, turret_func, atstmain_func, atst_side_alt_func,
    atst_side_main_func, tusk_shot_func, noghri_shot_func, clone_func, clone_alt_func,
    clone_pistol_alt_func, clone_commando_alt_func
missileDFlags FLAG FLAG ... ; => Damage flags. End the list with ";" as a SEPARATE word, on the same line:
    missileDFlags DAMAGE_HEAVY_WEAP_CLASS DAMAGE_DEATH_KNOCKBACK ;
- DAMAGE_RADIUS           : Damage was indirect
- DAMAGE_NO_ARMOR         : Armour does not protect
- DAMAGE_NO_KNOCKBACK     : No push, only view kick
- DAMAGE_NO_HIT_LOC       : Hit location does not change the damage
- DAMAGE_NO_PROTECTION    : Armour, shields, invulnerability and godmode have no effect
- DAMAGE_EXTRA_KNOCKBACK  : Extra push
- DAMAGE_DEATH_KNOCKBACK  : Push only when the target dies
- DAMAGE_IGNORE_TEAM      : Damages any team
- DAMAGE_NO_DAMAGE        : No damage, but the target reacts as if hit
- DAMAGE_DISMEMBER        : Can dismember
- DAMAGE_NO_KILL          : Damages, but does not kill
- DAMAGE_HEAVY_WEAP_CLASS : Heavy weapon damage; some objects only take damage with this flag
bounceWall 0/1     => 1: the projectile bounces on walls
bounceCount <int>  => Number of bounces before the projectile explodes / stops (with bounceWall 1)

// Charge (FL_BLASTER_CHARGED, FL_BOWCASTER, FL_BEAM_CHARGED)
chargeUnitTime <float>    => Time to charge one unit, in ms
maxChargeUnits <int>      => Maximum number of units
chargeMuzzleShader <path> => Shader drawn at the muzzle while charging
chargeMuzzleScale <float> => Size multiplier of chargeMuzzleShader, for big weapons

// Beam (FL_BEAM, FL_FULL_BEAM, FL_BEAM_CHARGED)
beamShader <path>         => Shader of the beam
beamColor <r> <g> <b>     => Colour of beamShader, each 0-1
fullBeamShader <path>     => Shader drawn on top at full charge / for FL_FULL_BEAM
fullBeamColor <r> <g> <b> => Colour of fullBeamShader, each 0-1
beamRadius <float>        => Thickness of the beam hit test, in units (0 = a line)
beamTrailEffect <path>    => Effect played along the beam
selfKnockback 0-1000      => Pushes the shooter backwards on each shot
```

```
    pushForce 0-1000          => Pushes the creature hit
    knockdownForce 0-1000     => Knocks the creature hit down (if it has knockdown animations)

    // Flamethrower (FL_FLAMETHROWER)
    effectDuration <ms> => How long the target burns (default 1000)

    // EMP explosion (FL_DEMP2_ALT)
    dempDetonateModel <path>  => Model of the explosion (default models/items/sphere.md3)
    dempDetonateShader <path> => Shader on that model (default gfx/effects/demp2shell)

    // Explosive projectile (FL_GRENADE, FL_IMPACT_GRENADE, FL_PROXIMITY_TRAP, FL_LASER_TRAP, missiles)
    explosionEffect <path> => Effect when it explodes
    shockwaveEffect <path> => Second effect played with the explosion
  }
}
```